



Department of Computer Science and Engineering
Midterm Examination Summer 2025
CSE 321: Operating Systems

Duration: 1 Hour 30 Minutes

Total Marks: 30

Answer all the following questions.
 Figures in the right margin indicate marks.

Name:	ID:	Sec:
-------	-----	------

1. Write the answers in the question paper. [1*2
=2]

CO1 a) Naima is designing a server on which the main process which first opens a file consisting of a huge chunk of data then creates child processes to handle the requests of different clients. While the child processes execute independently, the main server process may finish its execution first, without waiting for its child. So, some of the processes get disconnected from their parents completely. This may, in turn, cause some issues with resource management.

- I. **Identify** the issue that is present in Naima's system.
- II. **State** how it can be solved.

Answer:

I	
II	

- b)** Observe the following code. **Calculate** how many processes and how many threads will be created. [1*2
=2]

```
main(){
    a = fork();
    b = fork();
    if (a > 0){
        pthread_create(...);
        fork();
    }
}
```

Answer:

Processes	
Threads	

c) A parent process creates a shared memory segment. It generates a random value **3578** and stores it into the shared memory, then forks a child. The child process then generates a random value **4582** and saves it in the shared memory. The parent tries to read from the shared memory, it retrieves the value. **[1*2 =2]**

- I. **Identify** what value does the parent process retrieves
- II. **Justify** the reason behind the previous answer.

Answer:

I	
II	

d) Find outputs of the given code:

[1*5 =5]

```
int main() {
    int counter = 0;
    int i;

    for (i = 0; i < 2; i++) {
        pid_t pid = fork();

        if (pid < 0) {
            printf("Error forking.\n");
            return 1;
        }
        else if (pid == 0) {
            counter += 10;
            printf("Child created in loop %d: its counter is now %d.\n", i, counter);
            break;
        }
        else {
            wait(NULL);
            counter += 1;
        }
    }

    printf("Process from loop iteration %d has a final counter of %d.\n", i, counter);

    return 0;
}
```

Write the outputs in the question paper. The output sequence should be exactly matched.

Outputs	
1	
2	
3	
4	
5	

2. a) Write the answers in the question paper.

[1*2

CO1 Imagine a server environment where multiple interactive user applications =2]

(like web browsers, text editors, and chat clients) are being scheduled using the Round Robin (RR) algorithm. If the system administrator configures the time quantum to an extremely small value (e.g., 1-2 milliseconds).

- I. **What** is the direct consequence for the responsiveness of these applications?
- II. **How** does this affect the overall system efficiency from a user's perspective?

Answer:

I	
II	

b) Write the answer in the answer script.

Shoumo is implementing his OS. For his OS, he has decided to develop a **Multilevel Feedback Queue** as a scheduler. To design the scheduler, he decided to divide the ready queue into **three** levels. According to his preferences, in every level round-robin algorithm will be used, and some constraints should be maintained in each level, which are given below.

- **Q0:** time quantum = **3**, priority = **13**
- **Q1:** time quantum = **5**, priority = **26**
- **Q2:** time quantum = **7**, priority = **39**

Here, the lower priority value indicates the higher priority, and in case of an I/O request, the scheduler will promote a process by one level upon I/O operation completion. At a certain moment, Shoumo has collected information on burst times, arrival times, and I/O times of three processes.

Process	Burst Time	Arrival Time	I/O Time
P1	14	4	4 [After 6s of total CPU allocation]
P2	7	16	N/A
P3	15	0	2 [After 14s of total CPU allocation]

- I. **Draw** a Gantt chart using the multilevel feedback queue scheduling algorithm, showing the states of the ready queues of different levels. **[10]**
- II. **Calculate** the average waiting time and average turnaround time from the Gantt chart. **[1+1=2]**

3. a) Write the answers in the question paper.

[1*2

CO2 Consider the following code for the **compare_and_swap** hardware tool.

=2]

```
int compare_and_swap(int *target, int v1, int v2){
    int rv=*target;
    if(*target==v1){
        *target=v2;
    }
    return rv;
}

do{
    while(compare_and_swap(&lock,1,0)!=0)
        ; //do nothing

    //critical section

    lock=1;

    //remainder section
}while(true);
```

- I. **Identify** the error in the code that will create issues in synchronization.
- II. **Write** the fix for the error.

Answer:

I	
II	

b) Write the answers in the question paper.

[1]

Bob is developing a restaurant management application that handles table allocation using semaphores. The current implementation for managing 5 available tables is as follows:

```
table_sem = 5
available_tables = [1, 2, 3, 4, 5]
def seat_customer():
    wait(table_sem) // wait for a free table
    table = available_tables.pop() // assign a free table
    return table
def leave_customer(table):
    available_tables.append(table) // release the table
    signal(table_sem)
```

During concurrent usage, when multiple customers are seated or leave at the same time, logs show that the same table number is occasionally assigned to more than one customer, causing conflicts. As a developer, **identify** the issue and suggest a quick fix.

Answer:

--

c) Write the answers in the question paper.

[1*2

Consider the following code. According to this code, **find out** what values Thread 1 and Thread 2 will print. =2]

```
int array[] = {12, 67, 88, 34, 91, 5, 48, 73, 55, 29};

void* find_and_print_values(void* arg) {

    //if it is Thread 1 then print the elements in the first half of the array whose values are < 50
    //else print the elements in the second half of the array whose values are < 50

}

int main() {
    pthread_t threads[2];
    int i;

    for (i = 0; i < 2; i++) {
        pthread_create(&threads[i], NULL, find_and_print_values, (void*)(i + 1));
    }

    for (i = 0; i < 2; i++) {
        pthread_join(threads[i], NULL);
    }

    return 0;
}
```

Answer:

Thread 1	
Thread 2	